

Multi-vehicle Platoon

Multi-vehicle Platoon

1. Content Description
2. Preparation Work
 - 2.1 Site Requirements
 - 2.2 Navigation Map
 - 2.3 Implementation Principle
3. Program Startup
 - 3.1 Initialize Vehicle Pose
 - 3.2 Start Platoon Program
4. Core Code Analysis
 - 4.1 pub_follower_goal.py
 - 4.2 get_follower_point.py

1. Content Description

Run example programs to implement three vehicles arranged in preset formations. When the lead vehicle moves, the other two vehicles will also follow the movement and maintain the formation at the destination. The program has three optional formations:

- Column: Lead vehicle at the front, other two vehicles behind it, forming a queue similar to the Arabic numeral 1;
- Row: Lead vehicle in the middle, other two vehicles on its left and right sides, forming a queue similar to the Chinese character "—";
- Left-right escort: Lead vehicle in the middle, other two vehicles at its rear-left and rear-right, forming a queue similar to a "tank" shape.

For convenience of explanation, this tutorial uses robot1 to represent the lead vehicle, and robot2 and robot3 to represent the other two vehicles.

2. Preparation Work

This function requires 3 vehicles, and all 3 vehicles must be configured with namespaces and ROS_DOMAIN_ID.

2.1 Site Requirements

When running this function, choose as spacious a site as possible. Although there is obstacle avoidance between vehicles, if the site is not spacious enough, it will cause poor vehicle navigation effects or navigation planning failures.

2.2 Navigation Map

Before starting multi-vehicle platoon, you need to place the map files in the `~/map/` path of the vehicle that starts the map service. The map files include .yaml format parameter files and .pgm format image files.

2.3 Implementation Principle

This function relies on nav2 navigation when started. After giving robot1 a target point, robot1 will publish two TF transforms: the TF transform from robot1 to point2 and the TF transform from robot1 to point3. Point2 and point3 are the target points that robot2 and robot3 need to reach. During robot1's movement, it publishes two TF transforms. Robot2 and robot3 only need to obtain the positions of point2 and point3 on the map, then navigate to point2 and point3 respectively.

3. Program Startup

1. This section requires entering commands in the mainboard terminal. Choose to open the terminal according to the mainboard type. This course uses Raspberry Pi 5 as an example.
2. You need one vehicle to run the map service loading service and RVIZ visualization interface. This vehicle must place the map files in the `~/map/` path.
3. Run commands on the vehicle terminal that starts visualization and map service:

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

- **(ROBOT1)** (ORIN mainboard)

```
ros2 launch m3_bringup multi_nav2.launch.py start_mapserve:=True gui:=True
```

(RDK & PI5 mainboard) run in steps, visualization starts on the paired virtual machine `Rosmaster-M3-Ubuntu22.04`,

```
#Virtual machine side
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/multi_nav2.rviz
#Host side
ros2 launch m3_bringup multi_nav2.launch.py start_mapserve:=True gui:=True
use_rviz:=False
```

- Other vehicles **(ROBOT2/3)** directly run multi-vehicle navigation nodes without startup parameters

(ORIN mainboard)

```
ros2 launch m3_bringup multi_nav2.launch.py
```

(RDK & PI5 mainboard)

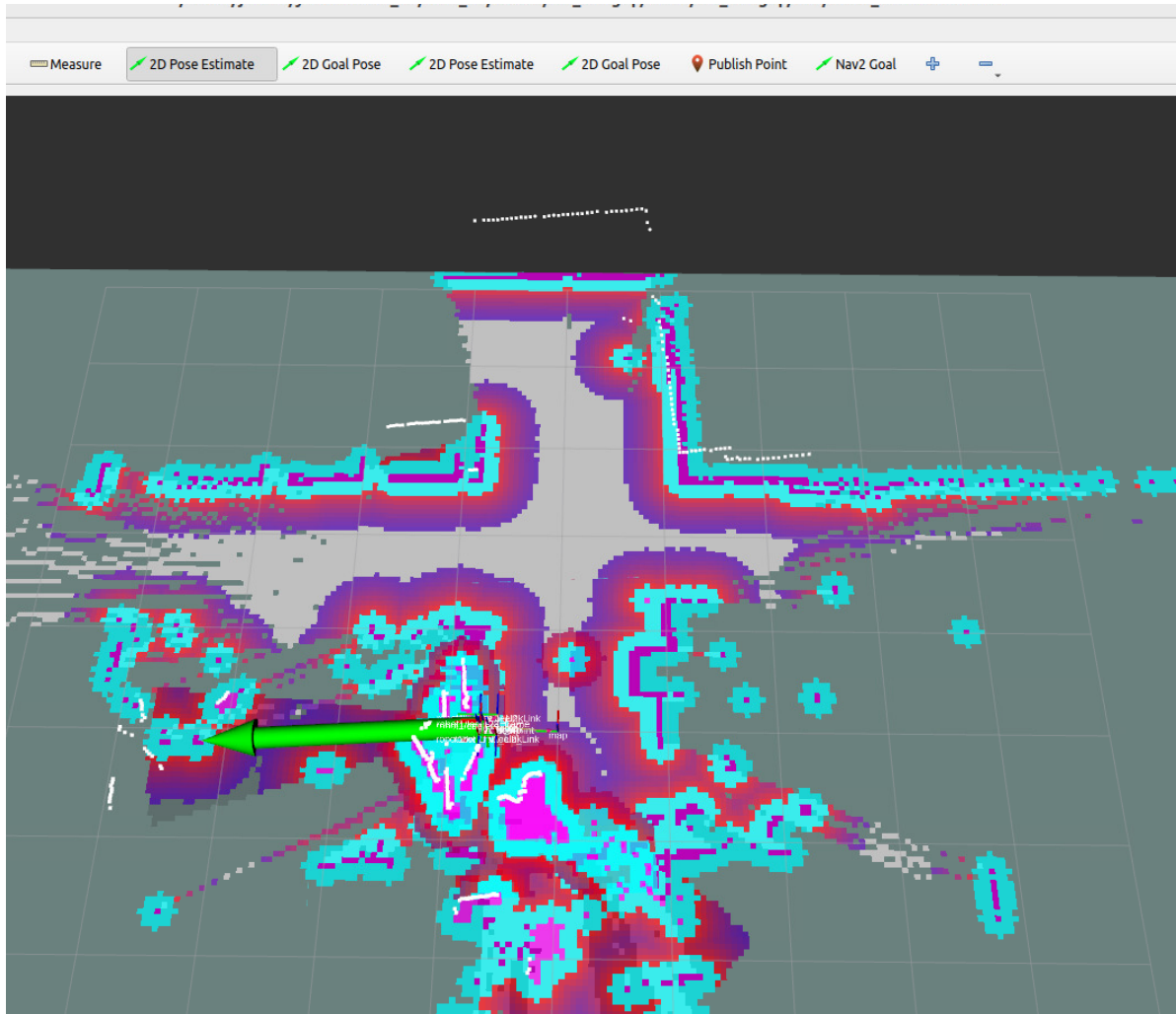
```
ros2 launch m3_bringup multi_nav2.launch.py use_rviz:=False
```

[!NOTE]

Optional startup parameter descriptions:

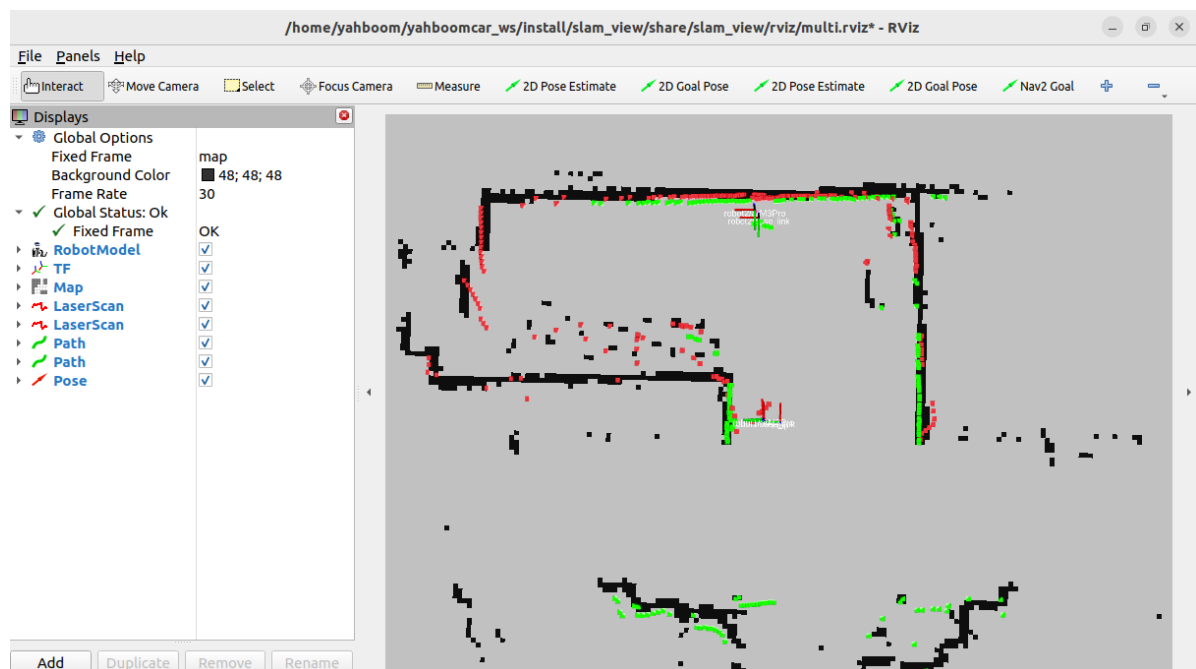
- start_mapserve: Start map server to load map, only one vehicle needs to start it, default `False`
- use_rviz: Start rviz2 visualization interface, only one vehicle needs to start it, default `False`
- gui: Start rtabmap visualization point cloud, default `False`
- map: Name of the map file to be loaded, map must be saved in the system `~/map/` path, default is `map.yaml`

3.1 Initialize Vehicle Pose



- Then, in rviz2, use the 【2D Pose Estimate】 tool to give the vehicle an initial pose. There are three 【2D Pose Estimate】 tools in rviz2, from left to right respectively for giving robot1 initial pose, robot2 initial pose, and robot3 initial pose. According to the actual positions of the three vehicles on the map, give initial poses to the three vehicles respectively.

As shown in the figure below, make the area scanned by the two radars overlap with the black areas on the map. Among them, green is the point cloud scanned by robot1 vehicle's radar, red is the point cloud scanned by robot2 vehicle's radar, and blue is the point cloud scanned by robot3 vehicle's radar.



3.2 Start Platoon Program

- On the vehicle side running RVIZ, enter the following command to start the TF publishing program:

```
ros2 run m3_common_demo pub_follower_goal
```

After the program runs, in rviz, use the first **【2D Goal Pose】** tool in rviz to give robot1 a target pose to navigate to that point. In rviz, you will see that according to the formation name (default is convoy, which is left-right escort), robot1/base_link points to point2 and point3.

- Then, on the vehicle side running RVIZ, enter the following command to start the robot2 and robot3 target point subscription and navigation program:

```
ros2 run m3_common_demo get_follower_point
```

After the program runs, robot2 and robot3 will navigate to the positions of point2 and point3 respectively. After robot2 and robot3 reach the target points, they will form a left-right escort formation with robot1. At this time, in rviz, use the first **【2D Goal Pose】** tool in rviz to give robot1 a target point. During robot1's navigation, robot2 and robot3 will also follow the movement.

4. Core Code Analysis

4.1 pub_follower_goal.py

Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/multi_robot/pub_follower_g  
oal.py
```

Initialization function,

```

def __init__(self):
    super().__init__('navigation_client')
    #Subscribe to /robot1/goal_pose topic, this topic will be published when
    using 【2D Goal Pose】 tool to give robot1 a target point
    self.get_goal_pose =
self.create_subscription(PoseStamped, "/robot1/goal_pose", self.get_GoalPoseCallBa
ck,1)
    #Initialize two static TF transform publishers
    self.robot1_to_point2_broadcaster = StaticTransformBroadcaster(self)
    self.robot1_to_point3_broadcaster = StaticTransformBroadcaster(self)

```

Topic callback function,

```

def get_GoalPoseCallback(self, msg):
    #Create TransformStamped data type to store the contents of two static
    transforms
    robot2_transform = TransformStamped()
    robot3_transform = TransformStamped()
    #Assign values to TF transform related data here. Here frame_id is
    robot1/base_link, child_frame_id is point2, indicating the published transform is
    the TF transform from robot1/base_link to point2
    robot2_transform.header.stamp = self.get_clock().now().to_msg()
    robot2_transform.header.frame_id = "robot1/base_link"
    robot2_transform.child_frame_id = "point2"

    robot3_transform.header.stamp = self.get_clock().now().to_msg()
    robot3_transform.header.frame_id = "robot1/base_link"
    robot3_transform.child_frame_id = "point3"

    #Determine the formation shape. column means vertical column, similar to
    Arabic numeral 1, so point2 is behind robot1/base_link, distance is -self.dist in
    x direction. Here self.dist defaults to 0.3, meaning point2 is 0.3 meters behind
    robot1/base_link; while point3 is 0.6 meters behind robot1/base_link,
    -2*self.dist in x direction.
    if self.queue == "column":
        robot2_transform.transform.translation.x = -self.dist
        robot2_transform.transform.translation.y = 0.0

        robot3_transform.transform.translation.x = -self.dist*2
        robot3_transform.transform.translation.y = 0.0

    #Determine the formation shape. row means horizontal row, similar to Chinese
    character "一", so point2 is to the left of robot1/base_link, distance is -
    self.dist in y direction. Here self.dist defaults to 0.3, meaning point2 is 0.3
    meters to the left of robot1/base_link; while point3 is 0.3 meters to the right
    of robot1/base_link, self.dist in y direction.
    elif self.queue == "row":
        robot2_transform.transform.translation.x = 0.0
        robot2_transform.transform.translation.y = -self.dist

        robot3_transform.transform.translation.x = 0.0
        robot3_transform.transform.translation.y = self.dist

```

```

#Determine the formation shape. convoy means left-right escort, similar to
tank, so point2 is to the rear-left of robot1/base_link, distance is -self.dist
in x direction and -self.dist in y direction. Here self.dist defaults to 0.3,
meaning point2 is 0.3 meters to the rear-left of robot1/base_link; while point3
is 0.3 meters to the rear-right of robot1/base_link, -self.dist in x direction
and self.dist in y direction.
elif self.queue == "convoy":
    robot2_transform.transform.translation.x = -self.dist
    robot2_transform.transform.translation.y = -self.dist

    robot3_transform.transform.translation.x = -self.dist
    robot3_transform.transform.translation.y = self.dist
#Here w is 1.0 indicating no rotation transform, only translation transform.
robot2_transform.transform.rotation.w = 1.0
robot3_transform.transform.rotation.w = 1.0
#Publish two static TF transforms
self.robot1_to_point2_broadcaster.sendTransform(robot2_transform)
self.robot1_to_point3_broadcaster.sendTransform(robot3_transform)
print("send TF.")

```

4.2 get_follower_point.py

In the virtual machine, code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/multi_robot/get_follower_point.py
```

Initialization function,

```

def __init__(self):
    super().__init__('tf_listener_node')
    # Create a TF 2 buffer and listener
    self.tf_buffer_p2 = tf2_ros.Buffer()
    self.tf_listener_p2 = tf2_ros.TransformListener(self.tf_buffer_p2, self)
    # Create a TF 2 buffer and listener
    self.tf_buffer_p3 = tf2_ros.Buffer()
    self.tf_listener_p3 = tf2_ros.TransformListener(self.tf_buffer_p3, self)

    #Define two publishers to publish target poses of robot2 and robot3
    self.pub_robot2_pose = self.create_publisher(PoseStamped,
"/robot2/goal_pose", 10)
    self.pub_robot3_pose = self.create_publisher(PoseStamped,
"/robot3/goal_pose", 10)

    self.p2_goal_pose = PoseStamped()
    self.p2_goal_pose.header.frame_id = "map"

    self.p3_goal_pose = PoseStamped()
    self.p3_goal_pose.header.frame_id = "map"

    # Create a timer (10Hz), get transform each time
    self.timer = self.create_timer(0.1, self.timer_callback)
    self.get_point2()
    self.get_point3()

```

Timer callback function,

```
def timer_callback(self):
    #Get transform between map and point2. The significance of this step is to
    get the pose of point2 on the map, then publish robot2's target pose
    self.get_point2()
    #Get transform between map and point3. The significance of this step is to
    get the pose of point3 on the map, then publish robot3's target pose
    self.get_point3()
```

Get point2 pose and publish robot2 target pose,

```
def get_point2(self):
    try:
        #Listen to TF transform between map and point2
        transform_p2 = self.tf_buffer_p2.lookup_transform('map', 'point2',
rcipy.time.Time())
        print("transform: ",transform_p2.transform.translation)
        print("-----")
        #Assign values to robot2 target data
        self.p2_goal_pose.pose.position.x = transform_p2.transform.translation.x
        self.p2_goal_pose.pose.position.y = transform_p2.transform.translation.y
        self.p2_goal_pose.pose.orientation.z = transform_p2.transform.rotation.z
        self.p2_goal_pose.pose.orientation.w = transform_p2.transform.rotation.w
        #Publish robot2 target point
        self.pub_robot2_pose.publish(self.p2_goal_pose)
    except (tf2_ros.TransformException, KeyError) as e:
        self.get_logger().warn(f"Could not transform: {e}")
```